1.

```
syedhanzalaali@syedhanzalaali: ~
syedhanzalaali@syedhanzalaali:~$ sudo apt-get update
[sudo] password for syedhanzalaali:
Hit:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,494 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,800 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [240 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9,932 B]
Get:10 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2,568 kB]
Get:11 http://pk.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [331 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [596 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [961 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [218 kB]
Get:16 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:17 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.5 kB]
Get:18 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2,735 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.5 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6,732 B]
Get:22 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:23 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [630 kB]
Get:24 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:25 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,558 kB]
Get:26 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [316 kB]
Get:27 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [385 kB]
Get:28 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [32.7 kB]
Get:29 http://pk.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [7,044 B]
Get:30 http://pk.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:31 http://pk.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,312 B]
Get:32 http://pk.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B]
Get:33 http://pk.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:34 http://pk.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 14.6 MB in 5s (2,990 kB/s)
```

2.

```
syedhanzalaali@syedhanzalaali:~$ sudo apt-get install libncurses5-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'libncurses-dev' instead of 'libncurses5-dev'
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
Suggested packages:
  ncurses-doc
The following NEW packages will be installed:
  libncurses-dev
0 upgraded, 1 newly installed, 0 to remove and 256 not upgraded.
Need to get 384 kB of archives.
After this operation, 2,417 kB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libncurses-dev amd64 6.4+20240113-1ubuntu2 [384 kB]
Fetched 384 kB in 1s (357 kB/s)
Selecting previously unselected package libncurses-dev:amd64.
(Reading database ... 153625 files and directories currently installed.)
Preparing to unpack .../libncurses-dev_6.4+20240113-1ubuntu2_amd64.deb ...
Unpacking libncurses-dev:amd64 (6.4+20240113-1ubuntu2) ...
Setting up libncurses-dev:amd64 (6.4+20240113-1ubuntu2) ...
Processing triggers for man-db (2.12.0-4build2) ...
```

3.

```
syedhanzalaali@syedhanzalaali:~$ sudo apt-get install libssl-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
Suggested packages:
  libssl-doc
The following NEW packages will be installed:
  libssl-dev
0 upgraded, 1 newly installed, 0 to remove and 256 not upgraded.
Need to get 2,407 kB of archives.
After this operation, 13.3 MB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 libssl-dev amd64 3.0.13-0ubuntu3.7 [2,407 kB]
Fetched 2,407 kB in 1s (1,618 kB/s)
Selecting previously unselected package libssl-dev:amd64.
(Reading database ... 153706 files and directories currently installed.)
Preparing to unpack .../libssl-dev_3.0.13-0ubuntu3.7_amd64.deb ...
Unpacking libssl-dev:amd64 (3.0.13-0ubuntu3.7) ...
Setting up libssl-dev:amd64 (3.0.13-0ubuntu3.7) ...
```

4.

```
root@syedhanzalaali:/kernels# rm linux-6.19.5.tar.xz
root@syedhanzalaali:/kernels# wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.19.5.tar.xz
--2026-03-01 22:19:17--  https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.19.5.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 199.232.81.176, 2a04:4e42:54::432
Connecting to cdn.kernel.org (cdn.kernel.org)|199.232.81.176|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 156053868 (149M) [application/x-xz]
Saving to: ‘linux-6.19.5.tar.xz’

linux-6.19.5.tar.xz                 100%[===================================================================================>] 148.82M  4.78MB/s    in 46s

2026-03-01 22:20:05 (3.20 MB/s) - ‘linux-6.19.5.tar.xz’ saved [156053868/156053868]
```

5.

```
root@syedhanzalaali:/kernels# tar xf linux-6.19.5.tar.xz
root@syedhanzalaali:/kernels# ls
linux-6.19.5  linux-6.19.5.tar.xz
root@syedhanzalaali:/kernels#
```

6.

```
root@syedhanzalaali:/kernels# cd linux-6.19.5
root@syedhanzalaali:/kernels/linux-6.19.5# ls
arch   certs    CREDITS  Documentation  fs       init      ipc     Kconfig  lib       MAINTAINERS  mm   README  samples  security  tools  virt
block  COPYING  crypto   drivers        include  io_uring  Kbuild  kernel   LICENSES  Makefile     net  rust    scripts  sound     usr
root@syedhanzalaali:/kernels/linux-6.19.5# mkdir helloworld
root@syedhanzalaali:/kernels/linux-6.19.5# cd helloworld
```

7.

```
Mar 1 22:49
root@syedhanzalaali: /kernels/linux-6.19.5
  GNU nano 7.2                                                    Kbuild

# A phony target that depends on all the preparation targets

PHONY += prepare
prepare: $(offsets-file) missing-syscalls $(atomic-checks)
        @:

# Ordinary directory descending
# ---------------------------------------------------------------------------

obj-y                   += init/
obj-y                   += usr/
obj-y                   += arch/$(SRCARCH)/
obj-y                   += $(ARCH_CORE)
obj-y                   += kernel/
obj-y                   += certs/
obj-y                   += mm/
obj-y                   += fs/
obj-y                   += ipc/
obj-y                   += security/
obj-y                   += crypto/
obj-y                   += helloworld/
obj-$(CONFIG_BLOCK)     += block/
obj-$(CONFIG_IO_URING)  += io_uring/
obj-$(CONFIG_RUST)      += rust/
obj-y                   += $(ARCH_LIB)
obj-y                   += drivers/
obj-y                   += sound/
obj-$(CONFIG_SAMPLES)   += samples/
obj-$(CONFIG_NET)       += net/
obj-y                   += virt/
obj-y                   += $(ARCH_DRIVERS)
obj-$(CONFIG_DRM_HEADER_TEST)   += include/
```

8.

```
455     common  futex_wait              sys_futex_wait
456     common  futex_requeue           sys_futex_requeue
457     common  statmount               sys_statmount
458     common  listmount               sys_listmount
459     common  lsm_get_self_attr       sys_lsm_get_self_attr
460     common  lsm_set_self_attr       sys_lsm_set_self_attr
461     common  lsm_list_modules        sys_lsm_list_modules
462     common  mseal                   sys_mseal
463     common  setxattrat              sys_setxattrat
464     common  getxattrat              sys_getxattrat
465     common  listxattrat             sys_listxattrat
466     common  removexattrat           sys_removexattrat
467     common  open_tree_attr          sys_open_tree_attr
468     common  file_getattr            sys_file_getattr
469     common  file_setattr            sys_file_setattr
470     common  listns                  sys_listns
471     common  helloworld              sys_helloworld
#
# Due to a historical design error, certain syscalls are numbered differently
# in x32 as compared to native x86_64.  These syscalls have numbers 512-547.
# Do not add new syscalls to this range.  Numbers 548 and above are available
```

9.

```
root@syedhanzalaa
GNU nano 7.2                                                    include/li
}

static inline unsigned int ksys_personality(unsigned int personality)
{
        unsigned int old = current->personality;

        if (personality != 0xffffffff)
                set_personality(personality);

        return old;
}

/* for __ARCH_WANT_SYS_IPC */
long ksys_semtimedop(int semid, struct sembuf __user *tsops,
                     unsigned int nsops,
                     const struct __kernel_timespec __user *timeout);
long ksys_semget(key_t key, int nsems, int semflg);
long ksys_old_semctl(int semid, int semnum, int cmd, unsigned long arg);
long ksys_msgget(key_t key, int msgflg);
long ksys_old_msgctl(int msqid, int cmd, struct msqid_ds __user *buf);
long ksys_msgrcv(int msqid, struct msgbuf __user *msgp, size_t msgsz,
                 long msgtyp, int msgflg);
long ksys_msgsnd(int msqid, struct msgbuf __user *msgp, size_t msgsz,
                 int msgflg);
long ksys_shmget(key_t key, size_t size, int shmflg);
long ksys_shmdt(char __user *shmaddr);
long ksys_old_shmctl(int shmid, int cmd, struct shmid_ds __user *buf);
long compat_ksys_semtimedop(int semid, struct sembuf __user *tsems,
                            unsigned int nsops,
                            const struct old_timespec32 __user *timeout);
long __do_semtimedop(int semid, struct sembuf *tsems, unsigned int nsops,
                     const struct timespec64 *timeout,
                     struct ipc_namespace *ns);

int __sys_getsockopt(int fd, int level, int optname, char __user *optval,
                int __user *optlen);
int __sys_setsockopt(int fd, int level, int optname, char __user *optval,
                int optlen);
asmlinkage long sys_helloworld(void);
#endif
```

10.

```
root@syedhanzalaali:/kernels/linux-6.19.5# ls /boot | grep config
config-6.14.0-37-generic
root@syedhanzalaali:/kernels/linux-6.19.5# cp /boot/config-6.14.0-37-generic .config
root@syedhanzalaali:/kernels/linux-6.19.5#
```

11.

```
Test module for mmiotrace (MMIOTRACE_TEST) [N/m/?] n
Test module to create a preempt / IRQ disable delay thread to test latency tracers (PREEMPTIRQ_DELAY_TEST) [N/m/?] n
Test module for in-kernel synthetic event generation (SYNTH_EVENT_GEN_TEST) [N/m/?] n
Test module for in-kernel kprobe event generation (KPROBE_EVENT_GEN_TEST) [N/m/?] n
Hist trigger debug support (HIST_TRIGGERS_DEBUG) [N/y/?] n
*
* Runtime Verification
*
Runtime Verification (RV) [Y/n/?] y
  Maximum number of per-task monitor (RV_PER_TASK_MONITORS) [2] (NEW)
  wwnr monitor (RV_MON_WWNR) [Y/n/?] y
  rtapp monitor (RV_MON_RTAPP) [N/y/?] (NEW)
  Runtime verification reactors (RV_REACTORS) [Y/n/?] y
    Printk reactor (RV_REACT_PRINTK) [Y/n/?] y
    Panic reactor (RV_REACT_PANIC) [Y/n/?] y

Sample kernel code

mple kernel code (SAMPLES) [Y/n/?] y
Build trace_events examples -- loadable modules only (SAMPLE_TRACE_EVENTS) [N/m/?] n
Build custom trace event example -- loadable modules only (SAMPLE_TRACE_CUSTOM_EVENTS) [N/m/?] n
Build trace_printk module - tests various trace_printk formats (SAMPLE_TRACE_PRINTK) [M/n/?] m
Build register_ftrace_direct() example (SAMPLE_FTRACE_DIRECT) [M/n/?] m
Build register_ftrace_direct() on multiple ips example (SAMPLE_FTRACE_DIRECT_MULTI) [N/m/?] n
Build custom ftrace ops example (SAMPLE_FTRACE_OPS) [N/m/y/?] n
Build sample module for kernel access to Ftrace instances (SAMPLE_TRACE_ARRAY) [M/n/?] m
Build kobject examples (SAMPLE_KOBJECT) [N/m/y/?] n
Build kprobes examples -- loadable modules only (SAMPLE_KPROBES) [N/m/?] n
Build kernel hardware breakpoint examples -- loadable module only (SAMPLE_HW_BREAKPOINT) [N/m/?] n
Build fprobe examples -- loadable modules only (SAMPLE_FPROBE) [N/m/?] n
Build kfifo examples -- loadable modules only (SAMPLE_KFIFO) [N/m/?] n
Build kdb command example -- loadable modules only (SAMPLE_KDB) [N/m/?] n
Build rpmsg client sample -- loadable modules only (SAMPLE_RPMSG_CLIENT) [N/m/?] n
Build live patching samples -- loadable modules only (SAMPLE_LIVEPATCH) [N/m/?] n
Build configfs patching sample -- loadable modules only (SAMPLE_CONFIGFS) [N/m/?] n
TSM measurement sample (SAMPLE_TSM_MR) [N/m/y/?] (NEW)
Build VFIO mtty example mediated device sample code (SAMPLE_VFIO_MDEV_MTTY) [N/m/?] n
Build VFIO mdpy example mediated device sample code (SAMPLE_VFIO_MDEV_MDPY) [N/m/?] n
Build VFIO mdpy example guest fbdev driver (SAMPLE_VFIO_MDEV_MDPY_FB) [N/m/y/?] n
Build VFIO mbochs example mediated device sample code (SAMPLE_VFIO_MDEV_MBOCHS) [N/m/?] n
Hung task detector test code (SAMPLE_HUNG_TASK) [N/m/y/?] (NEW)

configuration written to .config
```

12.

```
root@syedhanzalaali:/kernels/linux-6.19.5# make clean -j4
root@syedhanzalaali:/kernels/linux-6.19.5#
```

1. This step demonstrates the execution of the sudo apt-get update command, which refreshes the local package manager's list of available software and updates from the Ubuntu repositories.
2. The libncurses5-dev package is being installed here. This dependency provides the developer libraries necessary for generating text-based menu interfaces, which are used during the kernel configuration phase.
3. The libssl-dev package is being installed in this step. This library provides cryptographic functions that modern Linux kernels require to securely sign modules and certificates during the build process.
4. The official Linux kernel version 6.19.5 source code archive is being downloaded directly from the kernel.org repository using the wget command.
5. The downloaded linux-6.19.5.tar.xz archive is being extracted using the tar xf command to unpack all the necessary kernel source files into a usable directory.
6. After navigating into the newly extracted kernel directory, a new folder named helloworld is created. This directory is designated to hold the specific C code for the custom system call.
7. The main kernel Kbuild file is being modified in the nano text editor. The addition of obj-y += helloworld/ instructs the compiler to include the new custom directory when compiling the entire kernel.
8. The syscall_64.tbl file is being updated to officially register the custom system call with the operating system. The new call is named "helloworld" and is assigned the specific system call ID number 471.
9. The include/linux/syscalls.h header file is being edited to append the asmlinkage long sys_helloworld(void); function prototype at the very end of the document. This ensures the rest of the kernel recognizes the new function.
10. The existing boot configuration file from the current Ubuntu system (config-6.14.0-37-generic) is copied into the new kernel folder and renamed to .config so the new kernel inherits the correct hardware settings.
11. A configuration command is run to apply the copied settings to the new kernel environment, resulting in the successful writing of the updated parameters to the .config file.

12. The make clean -j4 command is executed to remove any stale temporary files, corrupted compilation data, or leftover object files to ensure a clean environment before attempting the main kernel build.

System Call C Source Code

This is the code for the actual system call logic, saved inside your new directory.

File : helloworld/helloworld.c

```
#include <linux/kernel.h>

#include <linux/syscalls.h>

SYSCALL_DEFINE(helloworld){

        Printk("hello world\n");

        Return 0;

}
```

System Call Makefile

This is the code for the local Makefile that tells the kernel how to compile your specific C file.

File : helloworld/Malefile

```
Obj-y := helloworld.o
```

13. The make bzImage command is executed to compile the core kernel image. This process translates the source code into a bootable compressed kernel file. Due to hardware resource limitations on the virtual machine, this specific compilation process could not be fully completed.

14. The make modules command is used to compile the loadable kernel modules, which act as necessary drivers for various hardware components and subsystems.

15. The make modules_install command is executed to install the previously compiled modules into the appropriate system directory so the new kernel can load them during runtime.

16. The make install command is run to copy the newly compiled kernel image, System.map, and configuration files into the system's /boot directory.

17. The sudo update-grub command is executed to update the system bootloader configuration, ensuring that the newly installed custom kernel appears as a selectable option in the GRUB startup menu.

18. The system is rebooted, and the uname -r command is used in the terminal to verify that the operating system successfully booted into the newly compiled custom kernel version containing the assigned roll number.

19. A user-space C program is written, compiled, and executed to invoke the custom system call using its assigned ID number (471) to verify its functional implementation within the new kernel.

Test Program Code

test_syscall.c

C

```c
#include <stdio.h>

#include <unistd.h>

#include <sys/syscall.h>

int main() {

    long int status = syscall(471);

    printf("System call returned %ld\n", status);

    return 0;

}
```

Conclusion

During the execution of this assignment, the final compilation of the kernel (make bzImage) could not be successfully completed due to severe hardware resource constraints and hypervisor instability within the Oracle VirtualBox environment. Despite incrementally increasing the virtual machine's allocated base memory to over 11GB and assigning 6 CPU cores to handle the heavy workload, the virtual machine consistently aborted the process with a "referenced memory could not be written" fatal error.
This physical memory allocation failure occurred during the highly intensive linking phase of the kernel build, preventing the generation of the final compressed boot image. Consequently, to maintain the stability of the host machine, the build process was safely halted. The subsequent installation, GRUB configuration, and verification phases have been documented theoretically to outline the complete standard operating procedure required to finalize the custom system call implementation.